

Report 8 – Diagnostica QALS

20 giugno 2026

1 Diagnostica di QALS

Fino ad ora i test su QALS sono stati svolti principalmente con l'obiettivo di verificare se, al variare delle configurazioni dei parametri, l'algoritmo fosse in grado di trovare soluzioni buone per il problema considerato. In particolare, l'interesse era quello di capire se QALS riuscisse a produrre soluzioni non troppo distanti da quella ottima. Tuttavia, per i test case di dimensione maggiore, i risultati ottenuti non sono stati sempre soddisfacenti. Per questo motivo si è deciso di introdurre una forma di diagnostica aggiuntiva, con lo scopo di osservare più nel dettaglio il comportamento dell'algoritmo durante la fase di ricerca.

L'idea consiste nell'utilizzare la distanza di Hamming tra configurazioni binarie. Nel contesto del problema Knapsack, una soluzione è rappresentata da un vettore binario:

$$z = (z_1, z_2, \dots, z_n),$$

dove:

$$z_i = \begin{cases} 1 & \text{se l'item } i \text{ è selezionato,} \\ 0 & \text{altrimenti.} \end{cases}$$

La distanza di Hamming tra due soluzioni binarie z e z' misura il numero di posizioni in cui esse differiscono:

$$d_H(z, z') = \sum_{i=1}^n \mathbf{1}_{z_i \neq z'_i}.$$

Nel nostro caso, questa misura permette di capire quanti item cambiano stato tra due soluzioni: un valore alto indica che molte variabili binarie sono diverse, mentre un valore basso indica che le due configurazioni sono simili.

2 Possibili confronti tramite distanza di Hamming

2.1 Distanza rispetto alla migliore soluzione trovata

Il primo confronto consiste nel calcolare la distanza tra la soluzione generata all'iterazione t , indicata con z_t , e la migliore soluzione trovata fino a quel momento, indicata con z_{best} :

$$d_H(z_t, z_{\text{best}}).$$

Questa misura permette di osservare se le soluzioni candidate generate dall'algoritmo tendono ad avvicinarsi o ad allontanarsi dalla migliore configurazione individuata durante la ricerca. Un valore alto indica che la soluzione corrente è distante dalla migliore soluzione trovata, mentre un valore basso indica che la soluzione generata è più vicina a z_{best} . Se questa distanza tende a diminuire, si può interpretare il comportamento dell'algoritmo come una fase di intensificazione attorno ad una regione dello spazio delle soluzioni.

2.2 Distanza rispetto alla soluzione corrente accettata

Il secondo confronto consiste nel calcolare la distanza tra la soluzione candidata z_t e la soluzione corrente accettata dall'algoritmo, indicata con z_{star} :

$$d_H(z_t, z_{\text{star}}).$$

In questo caso z_{star} non deve essere interpretata come la soluzione ottima, ma come lo stato corrente dell'algoritmo. Infatti, QALS può accettare anche soluzioni peggiorative con una certa probabilità, al fine di evitare di bloccarsi prematuramente in minimi locali. Questa distanza misura quindi l'ampiezza della mossa proposta nello spazio delle soluzioni. Rappresenta quindi una misura del movimento della ricerca rispetto alla configurazione attualmente accettata. Valori elevati indicano che l'algoritmo sta proponendo salti ampi nello spazio delle soluzioni, e quindi un comportamento più esplorativo. Valori bassi indicano invece che la nuova soluzione candidata è simile alla soluzione corrente, suggerendo una ricerca più locale attorno alla configurazione accettata.

2.3 Distanza rispetto alla soluzione ottima

Il terzo confronto consiste nel calcolare la distanza tra la soluzione generata z_t e la soluzione ottima nota, indicata con z_{opt} :

$$d_H(z_t, z_{\text{opt}}).$$

Questa misura indica quanto la soluzione generata sia distante dalla soluzione ottima del problema. Un valore alto indica che la soluzione candidata è lontana dall'ottimo, mentre un valore basso indica che essa è simile alla soluzione ottima. Se la distanza raggiunge valore zero, significa che QALS ha generato esattamente la soluzione ottima. Questo tipo di confronto è possibile solo nei casi in cui la soluzione ottima sia nota (nel nostro caso calcolata con Dynamic Programming).

3 Interpretazione generale

La distanza di Hamming può quindi essere utilizzata con significati diversi a seconda della soluzione di riferimento scelta. In particolare:

- $d_H(z_t, z_{\text{star}})$ misura l'ampiezza della mossa proposta rispetto alla soluzione corrente accettata;
- $d_H(z_t, z_{\text{best}})$ misura la distanza dalla migliore soluzione trovata fino a quel momento;
- $d_H(z_t, z_{\text{opt}})$ misura la distanza dalla soluzione ottima, quando questa è nota.

In questo modo, la distanza di Hamming non viene utilizzata come criterio di ottimizzazione, ma come strumento diagnostico per analizzare il comportamento della ricerca. Essa permette di distinguere tra fasi di esplorazione, caratterizzate da salti più ampi nello spazio delle soluzioni, e fasi di intensificazione, caratterizzate da soluzioni più vicine a una configurazione di riferimento.

4 Clustering

4.1 Motivazione

Durante l'esecuzione di QALS vengono generate diverse soluzioni candidate, indicate con z . Oltre a valutare il valore della funzione obiettivo $f_Q(z)$, risulta utile analizzare anche il modo in cui l'algoritmo esplora lo spazio delle soluzioni. A questo scopo si può utilizzare

una procedura di clustering diagnostico, il cui obiettivo non è modificare il comportamento dell'algoritmo, ma studiare le soluzioni prodotte durante la ricerca.

L'idea principale è raggruppare soluzioni simili tra loro. Poiché nel problema QUBO le soluzioni sono rappresentate come vettori binari, una misura naturale di distanza è la distanza di Hamming. In questo modo è possibile costruire una matrice delle distanze tra tutte le soluzioni generate da QALS. Tale matrice viene poi utilizzata come input per un algoritmo di clustering agglomerativo.

4.2 Procedura utilizzata

La procedura segue quattro passaggi principali. Prima si raccolgono tutte le soluzioni z generate durante l'esecuzione di QALS. Successivamente, per ogni coppia di soluzioni, viene calcolata la distanza di Hamming. I valori ottenuti vengono inseriti in una matrice simmetrica D , dove l'elemento $D[i][j]$ rappresenta la distanza tra la soluzione z_i e la soluzione z_j .

Una volta ottenuta la matrice delle distanze, viene applicato un algoritmo di clustering agglomerativo. L'algoritmo assegna quindi ogni soluzione a uno dei cluster individuati. Infine, una volta calcolato il numero di soluzioni appartenenti a ciascun cluster si considererà solo il cluster con cardinalità maggiore da cui si estrarrà il medoid. Il medoid è la soluzione appartenente al cluster che minimizza la distanza media di Hamming rispetto a tutte le altre soluzioni dello stesso cluster:

$$z_{\text{medoid}} = \arg \min_{z \in C_i} \frac{1}{|C_i| - 1} \sum_{\substack{z_j \in C_i \\ z_j \neq z}} d_H(z, z_j)$$

Questa analisi permette di capire se QALS tende a generare soluzioni appartenenti a una sola regione dello spazio di ricerca oppure se esplora regioni differenti ed inoltre capire su quali soluzioni si concentra la ricerca. Un numero ridotto di cluster molto popolati può indicare una concentrazione della ricerca in poche zone, mentre molti cluster piccoli possono suggerire una maggiore dispersione delle soluzioni generate.

4.3 Algoritmo

Algorithm 1 Clustering diagnostico delle soluzioni QALS con calcolo dei medoid

Input: *solutions_matrix*, *n_clusters*

Output: *labels*, *cluster_sizes*, *cluster_medoids*

```

1:  $Z \leftarrow []$ 
2: for ogni elemento item in solutions_matrix do
3:    $z \leftarrow item[0]$ 
4:   aggiungi  $z$  a  $Z$ 
5: end for
6:  $m \leftarrow |Z|$ 
7:  $D \leftarrow$  matrice  $m \times m$  inizializzata a 0
8: for  $i \leftarrow 0$  to  $m - 1$  do
9:   for  $j \leftarrow 0$  to  $m - 1$  do
10:     $D[i][j] \leftarrow d_H(Z[i], Z[j])$ 
11:   end for
12: end for
13: model  $\leftarrow$  AgglomerativeClustering(
    n_clusters = n_clusters,
    metric = "precomputed",
    linkage = "average")
14: labels  $\leftarrow$  model.fit_predict(D)
15: cluster_sizes  $\leftarrow \{\}$ 
16: cluster_medoids  $\leftarrow \{\}$ 
17: for ogni label in labels do
18:   if label non appartiene a cluster_sizes then
19:     cluster_sizes[label]  $\leftarrow 0$ 
20:   end if
21:   cluster_sizes[label]  $\leftarrow$  cluster_sizes[label] + 1
22: end for
23: largest_cluster  $\leftarrow \arg \max_{c \in \text{cluster\_sizes}} \text{cluster\_sizes}[c]$ 
24:  $C_{\max} \leftarrow \{Z[i] \mid \text{labels}[i] = \text{largest\_cluster}\}$ 
25: if  $|C_{\max}| > 1$  then
26:    $z_{\text{medoid}} \leftarrow \arg \min_{z \in C_{\max}} \frac{1}{|C_{\max}| - 1} \sum_{\substack{z_j \in C_{\max} \\ z_j \neq z}} d_H(z, z_j)$ 
27: else
28:    $z_{\text{medoid}} \leftarrow$  unico elemento di  $C_{\max}$ 
29: end if

```

4.4 Interpretazione dei risultati

L'output dell'algoritmo è composto da due elementi. Il primo è il vettore *labels*, che associa ogni soluzione al cluster corrispondente. Il secondo è *cluster_sizes*, che indica quante soluzioni appartengono a ciascun cluster.

Per esempio, se si ottiene:

$$labels = [0, 0, 1, 1, 1, 2]$$

allora le prime due soluzioni appartengono al cluster 0, le successive tre al cluster 1 e l'ultima al cluster 2. Di conseguenza, le dimensioni dei cluster sono:

$$cluster_sizes = \{0 : 2, 1 : 3, 2 : 1\}$$

.